COSC 39: Theory of Computation

Problem 1. Consider the following problem, called BOXDEPTH:

BOXDEPTH

- **Input:** A set of n axis-aligned squares R in the plane.
- **Output:** Size of the largest subset of squares in R containing a common point.

1. Describe a polynomial-time reduction from BOXDEPTH to MAXCLIQUE.
2. Describe a polynomial-time algorithm for BOXDEPTH. (No analysis of correctness is needed as long as the algorithm is indeed correct. Time analysis is required.)
3. Why don't these two results imply that $P = NP$?

Solution.

1. Given a BoxDepth instance, we create a MaxClique instance as follows:
   - Create a undirected graph G, where each node corresponds to a square in the BoxDepth instance.
   - We create an edge between two nodes iff they share a common point.

   Let $|R| = n$. This reduction is polynomial time since we create at most n nodes and $\binom{n}{2}$ edges.

   ($\Longrightarrow$)

   yes-instance of BOXDEPTH gives a yes-instance of MAXCLIQUE since the largest subset of squares in R containing a common point would correspond to a clique in G. This is the largest clique since by construction we only have edges between nodes if they share a common point.

   ($\Longleftarrow$)

   yes-instance of MAXCLIQUE gives a yes-instance of BOXDEPTH since the largest clique in G implies all the squares corresponding to the nodes in the clique share a common point and once again we know this is the largest subset by construction.

2. Since there are at most $4|R|$ points, bruteforce checking how many squares share a given point and sorting the array would give us a polynomial time algorithm.

3. This doesn't imply $P = NP$ since we reduce to a 'harder' problem. So solving MaxClique in P time would imply we could solve BoxDepth in P time but the converse is not true.

Problem 2. Suppose you are given a magic black box that can determine (in an instant), given an arbitrary set X of positive integers, whether X can be partitioned into two sets A and B such that

$$\sum_{a \in A} a = \sum_{b \in B} b.$$

Describe and analyze a polynomial-time algorithm that either computes an equal partition of a given set of positive integers, or correctly reports that no such partition exists, using the magic black box as a subroutine.

Solution.

Given a set X we ask the box if there exists an equal partition of X.

1. If the box says no, we say no and we are done.
2. If the box says yes, we do the following,
   (a) We define singleton groups g_i such that each group corresponds to a number in the set.
   (b) Pick some group g, then loop over the other groups h and check if the set X_i with $g + h$ instead of g, h still has a valid partition. If it does, repeat this process, if it doesn't for any h then the partitions are g and $X \setminus g$.

Let $|X| = n$. Then we have at most $n - 1$ runs of the algorithm described in step 2. And for each run, for a fixed g there are at most $|X_i| - 1$ oracle queries (this is bounded by $n - 1$). So we have $O((n-1)^2) = O(n^2)$ which is polynomial.

Problem 3. Prove that the following language is undecidable:

$$L = \left\{ \langle M, w \rangle : \begin{array}{l} M \text{ is a Turing machine with alphabet } \{0,1\}, \\ \text{and at one step of } M(w), \text{ some empty cell } \square \\ \text{changes to } 0 \end{array} \right\}.$$

Solution.

We reduce from the halting problem

$$\text{HALT} = \{\langle M, w \rangle : M \text{ halts on input } w\},$$

which is undecidable.

The reduction. Given $\langle M, w \rangle$, we construct a Turing machine M' with alphabet $\{0, 1\}$ such that on input w, the machine M' does the following:

1. *Simulate M on w step by step.* Throughout the simulation M' maintains the invariant that *whenever it needs to use a cell that is currently empty ($\square$), it first writes a 1 into that cell, and only afterwards may overwrite it.* Hence during the simulation every blank cell that is touched is changed $\square \to 1$ first, and no cell is ever changed $\square \to 0$.

2. *If M halts*, the simulation reaches a halting configuration of M. At that point M' scans right past all the cells it has used until it reaches an empty cell, writes a 0 there (changing $\square \to 0$), and halts.

Correctness.

($\implies$)

Suppose $\langle M, w \rangle \in \text{HALT}$, so M halts on w. Then the simulation in step 1 terminates, M' executes step 2, and at that step an empty cell is changed to 0. Hence $\langle M', w \rangle \in L$.

($\impliedby$)

Suppose $\langle M, w \rangle \notin \text{HALT}$, so M runs forever on w. Then M' never finishes step 1 and never executes step 2. By the invariant, no step of the simulation changes a cell from $\square$ to 0. Therefore no step of $M'(w)$ ever changes an empty cell to 0, so $\langle M', w \rangle \notin L$. Combining the two directions,

$$\langle M, w \rangle \in \text{HALT} \iff \langle M', w \rangle \in L,$$

so f is a valid mapping reduction $\text{HALT} \leq_m L$. Since HALT is undecidable, L is undecidable.

Problem 4. Let G be an undirected graph. A subset S of vertices in G is *half-independent* if each vertex in S is adjacent to at most one other vertex in S.
Prove that finding the size of the largest half-independent set of vertices in a given undirected graph is NP-complete (that is, NP-hard and in NP).

Solution.

We use the decision version: given G and an integer k, does G have ahalf-independent set of size $\geq k$? Call this problem HALFIS.**HALFIS $\in$ NP.** A half-independent set S with $|S| \geq k$ is acertificate: we check $|S| \geq k$ and, for each $v \in S$, count its neighbors in$S$ and verify the count is at most 1. This runs in $O(|V| + |E|)$ time.**HALFIS is NP-hard.** We reduce from INDEPENDENTSET(NP-hard): given $G = (V, E)$ and k, decide whether G has an independent set ofsize $\geq k$.*Construction.* Let $n = |V|$. Form H by attaching one new pendant leafℓ_v to each $v \in V$:

$$V(H) = V \cup L, \quad L = \{\ell_v : v \in V\}, \qquad E(H) = E \cup \{\, v\ell_v : v \in V \,\}.$$

Output the instance $(H, k + n)$. H has $2n$ vertices and $|E| + n$ edges, so it isbuilt in polynomial time. Let $\alpha(G)$ be the maximum independent-set size in G.We show the maximum half-independent set of H has size exactly $n + \alpha(G)$.*($\geq$).* Let I be a maximum independent set of G and set $S = I \cup L$, so$|S| = n + \alpha(G)$. Each leaf ℓ_v has only the neighbor v, hence at most oneneighbor in S. Each $v \in I$ has neighbors $N_G(v) \cup \{\ell_v\}$ in H; as I isindependent, $N_G(v) \cap S = \emptyset$, so v's only neighbor in S is ℓ_v.Thus S is half-independent.*($\leq$).* Let S be a *maximum* half-independent set of H.*No edge of G lies inside S.* Suppose $u, v \in S$ with $uv \in E$. Then $u, v$are each other's single allowed neighbor in S, so $\ell_u, \ell_v \notin S$.Replace S by $S' = (S \setminus \{u\}) \cup \{\ell_u, \ell_v\}$. Now v's only neighborin S' is ℓ_v; ℓ_v has only neighbor v; ℓ_u has neighbor $u \notin S'$;and no other vertex gains a neighbor. So S' is half-independent with $|S'| = |S| + 1$,contradicting maximality. Hence $S \cap V$ is independent in G.*All leaves lie in S.* If $\ell_v \notin S$, then since $S \cap V$ isindependent, v has no neighbor in S besides ℓ_v; adding ℓ_v keeps Shalf-independent and increases its size, contradicting maximality.Thus $S = L \cup (S \cap V)$ with $S \cap V$ independent, so $|S| = n + |S \cap V| \leq n + \alpha(G)$.Combining the bounds, H's maximum half-independent set has size exactly$n + \alpha(G)$, so

$$\alpha(G) \geq k \iff H \text{ has a half-independent set of size } \geq n + k,$$

which is the produced instance. Hence INDEPENDENTSET $\leq_p$ HALFIS,so HALFIS is NP-hard. Being in NP and NP-hard, HALFIS is NP-complete.